

Reviewer Pack — Permanent-Variance of NW Design Matrix Predicts PRG Bias

Entry 92f05cc72075 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 02:42:44 UTC

Permanent-Variance of NW Design Matrix Predicts PRG Bias

Entry ID: 92f05cc72075 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 02:42:44 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics of 0/1 matrix permanents (Ryser-formula variance over random row-sign perturbations) **Field B** (complexity object): Nisan-Wigderson pseudorandom generator output bias against small DNF distinguishers

Statement:

Let D be an (ℓ, a) -NW design on n variables encoded as an $m \times n$ 0/1 incidence matrix M (rows = subsets S_i , columns = universe). Define $V(M) := \log_2(1 + \text{Var}_{\{\epsilon_j \in \{+/-1\}^n\}}[\text{perm}(M_{\epsilon})]) / m$, where M_{ϵ} is M with column j scaled by ϵ_j and perm is the 0/1 permanent on the $(m \times m)$ submatrix induced by the m heaviest columns. We conjecture that for every hard truth table $f : \{0,1\}^\ell \rightarrow \{0,1\}$ of formula-leaf-count $L(f) \geq 2^{\ell/3}$, the maximum bias of $\text{NW}_{\{D,f\}}$ against width- w DNF distinguishers satisfies $\text{bias}(\text{NW}_{\{D,f\}}, w) \leq 2^{-c \cdot V(M) \cdot m / w}$ for an absolute constant $c > 0$, so $V(M)$ is a constructive lower bound certificate for PRG quality.

Rationale (proposer's reasoning):

Permanent variance over sign perturbations measures how 'spread' the design's row-overlap pattern is, which is exactly the combinatorial quantity controlling pairwise-intersection cancellation in the NW hybrid argument; existing analyses use only $|S_i \cap S_j| \leq a$, but the variance aggregates higher-order interference. Linking permanent fluctuation (a 0/1 matrix invariant computable in $O(m \cdot 2^m)$ by Ryser) to NW bias is, to our knowledge, unstudied and sidesteps natural-proofs because the invariant depends on the design only, not on f 's truth table.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c80c249328a8024f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all tested $(\ell \in \{3,4,5\}, m \in \{\ell+1..12\}, a \in \{1,2\})$ configurations with ≥ 20 random NW designs per config and hard f 's ($L(f) \geq 2^{\ell/3}$), check $\log_2(1/\text{bias}(\text{NW}_{\{D,f\}}, w))$

$\geq 0.05 * V(M) * m / w$ for w in $\{1,2,3\}$. SUPPORTED iff $\geq 80\%$ of (D,f,w) pairs satisfy it AND no single pair violates by margin $> 2x$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.70	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Nisan-Wigderson pseudorandom generator DNF bias design matrix - permanent variance 0/1 matrix Ryser formula pseudorandomness - combinatorial design PRG fooling DNF lower bound certificate

Top relevant hits considered: - [<http://arxiv.org/abs/1311.6531v3>] Brains and pseudorandom generators - [<http://arxiv.org/abs/2412.05161v1>] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [<http://arxiv.org/abs/2404.12647v1>] Simple constructions of linear-depth t-designs and pseudorandom unitaries - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/2012.01667v2>] Computing the matrix fractional power with the double exponential formula - [<http://arxiv.org/abs/1503.00822v2>] Product Ranks of the 3×3 Determinant and Permanent - [<http://arxiv.org/abs/2211.03396v3>] Certificate Games and Consequences for the Classical Adversary Bound - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import itertools

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(n):
            if i != j:
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    return [[A[i][j] / A[i][i] for j in range(n)] for i in range(m)]

def determinant(A):
```

```

m, n = len(A), len(A[0])
if m != n:
    raise ValueError("Matrix must be square")
if n == 1:
    return A[0][0]
det = 0
for j in range(n):
    submatrix = [row[:j] + row[j+1:] for row in A[1:]]
    det += (-1) ** j * A[0][j] * determinant(submatrix)
return det

def permanent(M):
    m, n = len(M), len(M[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 0:
        return 1
    if n == 1:
        return M[0][0]
    perm = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in M[1:]]
        sign = (-1) ** (n - 1 - j)
        perm += sign * M[0][j] * permanent(submatrix)
    return perm

def run_trial(seed: int) -> dict:
    random.seed(seed)

    ell = random.choice([3, 4, 5])
    m = random.randint(ell + 1, 12)
    a = random.choice([1, 2])
    D = [[random.choice([0, 1]) for _ in range(m)] for _ in range(ell)]

    # Compute V(M) via Ryser's formula on the m heaviest columns
    M_heaviest = sorted(D, key=lambda row: sum(row), reverse=True)[:m]
    perm_var = permanent([[M_heaviest[i][j] * M_heaviest[j][i] for j in range(m)] for i in range(m)])
    V_M = math.log2(1 + perm_var) / m

    # Pick a hard function f
    def is_hard(f):
        return len(list(itertools.product([0, 1], repeat=ell))) >= 2 ** (ell // 3)

    for _ in range(64):
        eps = [random.choice([-1, 1]) for _ in range(m)]
        M_eps = [[M_heaviest[i][j] * eps[j] for j in range(m)] for i in range(m)]
        perm_var_eps = permanent([[M_eps[i][j] * M_eps[j][i] for j in range(m)] for i in range(m)])
        V_M_eps = math.log2(1 + perm_var_eps) / m
        if abs(V_M - V_M_eps) > 0.1:
            return {
                "metric_name": "V(M)",
                "metric_value": V_M,
                "instances_tested": 64,
                "conjecture_holds": False,
                "counterexample": "Ryser's formula discrepancy"
            }

```

```

# Estimate bias(NW_{D,f}, w) by enumerating all width-w≤3 DNFs on m variables
def NW(D, f):
    return sum(f(tuple(row)) for row in D)

def max_bias(w):
    max_b = 0
    for dnf in itertools.product([0, 1], repeat=m):
        if sum(dnf) == w:
            bias = abs(NW(D, lambda x: int(all(x[i] == dnf[i] for i in range(m)))) - 0.5)
            max_b = max(max_b, bias)
    return max_b

for w in [1, 2, 3]:
    bias = max_bias(w)
    if math.log(1 / bias) < 0.05 * V_M * m / w:
        return {
            "metric_name": "bias(NW_{D,f}, w)",
            "metric_value": bias,
            "instances_tested": 64,
            "conjecture_holds": False,
            "counterexample": f"Failed for width {w}"
        }

return {
    "metric_name": "V(M)",
    "metric_value": V_M,
    "instances_tested": 64,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08ccc640.py", line 123, in <module>
    result = run_trial(seed)
             ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_08ccc640.py", line 66, in run_trial
    perm_var = permanent([[M_heaviest[i][j] * M_heaviest[j][i] for j in range(m)] for i in range(m)])
                        ~~~~~^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any data, so no support fraction or violation ratio could be computed against the pre-registered criterion. | next: Fix the indexing bug in the permanent computation (the M_heaviest submatrix access uses M_heaviest[j][i] where j ranges over m but rows are indexed differently) and rerun the trials.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	21368
2	preregistration	claude_max	opus	0	0	7162
3	novelty	claude_max	opus	0	0	3058
4	novelty	claude_max	opus	0	0	5853
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13276
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15429
7	judge	claude_max	opus	0	0	4019

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 70166 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/92f05cc72075.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/92f05cc72075.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/92f05cc72075.tar.gz` (if generated)